

SPRING DATA JPA

Complete Interview Preparation Guide

Java / Spring Ecosystem

What's Inside

60+ Interview Questions & Detailed Answers
Code Examples & Best Practices
Quick Reference Cheat Sheet
Frequently Asked Questions Highlighted

★ Frequently asked questions are marked with a red star

Table of Contents

Table of Contents.....	2
Section 1: Introduction to Spring Data JPA.....	3
Section 2: JPA Entities and Annotations.....	4
Section 3: Entity Relationships.....	5
Section 4: Spring Data JPA Repositories.....	7
Section 5: Transactions in Spring Data JPA.....	10
Section 6: JPA Entity Lifecycle and Caching.....	11
Section 7: Spring Data JPA Auditing.....	12
Section 8: Criteria API and Specifications.....	13
Section 9: Projections and DTOs.....	13
Section 10: Inheritance Mapping Strategies.....	14
Section 11: Custom Repository Implementations.....	15
Section 12: Performance and Best Practices.....	15
Section 13: Schema Management and Configuration.....	17
Section 14: Advanced Topics.....	17
Section 15: Scenario-Based Questions.....	19
Spring Data JPA — Quick Reference Cheat Sheet.....	21
Core Repository Hierarchy.....	21
Common Annotations.....	21
Query Method Keywords.....	21
Transaction Propagation Quick Reference.....	22
application.properties Key Settings.....	22
N+1 Problem — Quick Fix Reference.....	22

Section 1: Introduction to Spring Data JPA

★FREQUENTLY ASKED Q1: What is Spring Data JPA?

Answer:

Spring Data JPA is a part of the larger Spring Data family that simplifies the implementation of JPA-based (Java Persistence API) data access layers. It reduces boilerplate code required to implement data access layers for various persistence stores.

Key points:

- Built on top of JPA (typically Hibernate as the provider)
- Provides repository abstractions to eliminate repetitive DAO code
- Supports query derivation from method names
- Part of the Spring Data umbrella project
- Enables CRUD operations with minimal configuration

★FREQUENTLY ASKED Q2: What is the difference between JPA, Hibernate, and Spring Data JPA?

Answer:

These three are often confused. Here is the distinction:

Technology	Type	Purpose
JPA	Specification (JSR-338)	Defines ORM standard for Java
Hibernate	Implementation (JPA provider)	Implements JPA spec, maps objects to DB
Spring Data JPA	Abstraction layer (on top of JPA)	Simplifies repository creation & queries

★FREQUENTLY ASKED Q3: What are the core interfaces in Spring Data JPA?

Answer:

Spring Data JPA provides a hierarchy of repository interfaces:

- `Repository<T, ID>` — Marker interface, root of the hierarchy
- `CrudRepository<T, ID>` — Provides CRUD operations (save, findById, findAll, delete, etc.)
- `PagingAndSortingRepository<T, ID>` — Adds pagination and sorting
- `JpaRepository<T, ID>` — Extends `PagingAndSortingRepository`, adds JPA-specific methods like `flush()`, `saveAndFlush()`, `deleteInBatch()`

Example declaration:

```
public interface UserRepository extends JpaRepository<User, Long> { }
```

Q4: What dependencies are required for Spring Data JPA in a Spring Boot project?

Answer:

Maven dependency in pom.xml:

```
<dependency>
```

```
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>com.h2database</groupId>  <!-- or MySQL, PostgreSQL -->
  <artifactId>h2</artifactId>
</dependency>
```

Gradle (build.gradle):

```
implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
runtimeOnly 'com.h2database:h2'
```

Section 2: JPA Entities and Annotations

★ FREQUENTLY ASKED Q5: What is a JPA Entity? What annotations are used to define one?

Answer:

A JPA entity is a lightweight, persistent domain object that is mapped to a database table.

Core annotations:

- `@Entity` — Marks class as a JPA entity
- `@Table(name = "users")` — Specifies the table name (optional)
- `@Id` — Marks the primary key field
- `@GeneratedValue` — Configures primary key generation strategy
- `@Column` — Customizes column mapping (name, nullable, length, etc.)

Example Entity:

```
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "user_name", nullable = false, length = 100)
    private String username;

    @Column(nullable = false, unique = true)
    private String email;
    // getters and setters
}
```

★ FREQUENTLY ASKED Q6: What are the GenerationType strategies for @GeneratedValue?

Answer:

There are four strategies:

- `GenerationType.AUTO` — JPA selects the appropriate strategy (default)
- `GenerationType.IDENTITY` — Uses auto-increment column (MySQL, PostgreSQL SERIAL)

- GenerationType.SEQUENCE — Uses DB sequence; requires @SequenceGenerator; best for Oracle
- GenerationType.TABLE — Uses a separate table to store the next value; portable but slow

Best practice: Use IDENTITY for MySQL/PostgreSQL, SEQUENCE for Oracle.

Q7: What is the difference between @Column(nullable = false) and @NotNull?

Answer:

- @Column(nullable = false) — DDL-level constraint; affects schema generation (creates NOT NULL in DB)
- @NotNull (Bean Validation) — Application-level validation; triggers before persistence but does not generate DDL

Best practice: Use both together for comprehensive validation.

Q8: What are Embedded and Embeddable in JPA?

Answer:

@Embeddable marks a class whose instances are stored as part of an owning entity (no separate table). @Embedded is used in the entity to embed an @Embeddable object.

```
@Embeddable
public class Address {
    private String street;
    private String city;
}

@Entity
public class Employee {
    @Id private Long id;
    @Embedded
    private Address address;
}
```

Q9: What is @Transient in JPA?

Answer:

@Transient marks a field that should NOT be persisted to the database. The field is ignored during mapping.

```
@Transient
private String tempPassword; // not saved to DB
```

Section 3: Entity Relationships

★FREQUENTLY ASKED Q10: Explain the four types of JPA relationships with examples.

Answer:

- @OneToOne — One entity maps to exactly one other entity
- @OneToMany — One entity maps to many instances of another entity
- @ManyToOne — Many entities map to one entity
- @ManyToMany — Many entities relate to many entities (uses join table)

@OneToMany / @ManyToOne Example:

```
@Entity public class Department {
    @Id private Long id;
    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL)
    private List<Employee> employees;
}

@Entity public class Employee {
    @Id private Long id;
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "dept_id")
    private Department department;
}
```

★FREQUENTLY ASKED Q11: What is the difference between FetchType.LAZY and FetchType.EAGER?

Answer:

- FetchType.EAGER — Related entities are loaded immediately with the parent entity (one SELECT or JOIN)
- FetchType.LAZY — Related entities are loaded only when accessed (proxy is created, load on first access)

Defaults:

- @OneToMany, @ManyToMany — LAZY by default
- @ManyToOne, @OneToOne — EAGER by default

Best practice: Prefer LAZY loading to avoid performance issues. Use JOIN FETCH in JPQL when you need related data.

★FREQUENTLY ASKED Q12: What is CascadeType and what are the available types?

Answer:

Cascade types determine which JPA lifecycle operations on a parent entity are propagated to child entities.

- CascadeType.PERSIST — Save parent also saves children
- CascadeType.MERGE — Merging parent also merges children
- CascadeType.REMOVE — Deleting parent deletes children
- CascadeType.REFRESH — Refreshing parent refreshes children
- CascadeType.DETACH — Detaching parent detaches children
- CascadeType.ALL — Applies all of the above

Caution: Avoid ALL + REMOVE on @ManyToMany — can lead to unintended mass deletes.

★FREQUENTLY ASKED Q13: What is the N+1 problem in JPA and how do you solve it?**Answer:**

The N+1 problem occurs when fetching N parent entities results in N additional queries to fetch their related entities, instead of a single query.

Example of the problem: Fetching 100 orders and then accessing each order's items triggers 100 additional SELECT queries.

Solutions:

- JOIN FETCH in JPQL: `@Query("SELECT o FROM Order o JOIN FETCH o.items")`
- `@EntityGraph` to specify which associations to load eagerly
- Batch fetching using `@BatchSize(size = 20)`
- Using projections or DTOs to fetch only required fields

JOIN FETCH Example:

```
@Query("SELECT d FROM Department d JOIN FETCH d.employees WHERE d.id = :id")
Department findByIdWithEmployees(@Param("id") Long id);
```

Q14: What is orphanRemoval in JPA?**Answer:**

`orphanRemoval = true` removes child entities from the database when they are removed from the parent's collection (even without `CascadeType.REMOVE`).

```
@OneToMany(mappedBy = "dept", orphanRemoval = true)
private List<Employee> employees;
```

Difference from `CascadeType.REMOVE`: `orphanRemoval` acts on collection removal, `REMOVE` acts when the parent entity itself is deleted.

Section 4: Spring Data JPA Repositories

★FREQUENTLY ASKED Q15: How does Spring Data JPA implement repository interfaces at runtime?**Answer:**

Spring Data JPA uses Spring AOP and Java dynamic proxies to create implementations at runtime. When the application context starts up, Spring scans for interfaces extending `Repository` and generates proxy implementations using `SimpleJpaRepository` as the base.

- `@EnableJpaRepositories` triggers repository discovery
- Spring creates a proxy bean for each repository interface
- `SimpleJpaRepository` provides default CRUD implementations using `EntityManager`

★FREQUENTLY ASKED Q16: What is Query Derivation (Derived Query Methods) in Spring Data JPA?**Answer:**

Spring Data JPA can automatically generate queries by parsing method names in repositories. The method name is broken into subject (find, count, delete) and criteria parts.

Examples:

```
// SELECT * FROM user WHERE email = ?
User findByEmail(String email);

// SELECT * FROM user WHERE last_name = ? ORDER BY first_name ASC
List<User> findByLastNameOrderByFirstNameAsc(String lastName);

// SELECT * FROM user WHERE age > ? AND active = true
List<User> findByAgeGreaterThanAndActiveTrue(int age);

// SELECT COUNT(*) FROM user WHERE status = ?
long countByStatus(String status);
```

Supported keywords: And, Or, Is, Equals, Between, LessThan, GreaterThan, Like, NotLike, In, NotIn, True, False, OrderBy, Not, IsNull, IsNotNull, Containing, StartingWith, EndingWith

★FREQUENTLY ASKED Q17: What is the @Query annotation and when should you use it?**Answer:**

@Query allows writing custom JPQL or native SQL queries on repository methods when derived query names become too complex.

JPQL Query:

```
@Query("SELECT u FROM User u WHERE u.email = :email AND u.status = :status")
User findByEmailAndStatus(@Param("email") String email,
                           @Param("status") String status);
```

Native SQL Query:

```
@Query(value = "SELECT * FROM users WHERE email = ?1", nativeQuery = true)
User findByEmailNative(String email);
```

When to use @Query:

- Query logic is too complex for method name derivation
- Joining multiple tables
- Aggregation functions
- Using specific DB features (native)

Q18: What is the difference between JPQL and native SQL in Spring Data JPA?**Answer:**

- JPQL (Java Persistence Query Language) — Works on entity objects; database-agnostic; uses entity class and field names
- Native SQL — Works on actual DB tables and columns; database-specific; can use all DB features

Use JPQL for portability. Use nativeQuery = true only when specific DB features are needed.

★FREQUENTLY ASKED Q19: How do you implement Pagination and Sorting?**Answer:**

Spring Data JPA provides built-in pagination through Pageable and Sort interfaces.

```
// Repository
Page<User> findByStatus(String status, Pageable pageable);

// Service
Pageable pageable = PageRequest.of(0, 10, Sort.by("lastName").ascending());
Page<User> page = userRepo.findByStatus("ACTIVE", pageable);

// Access results
List<User> users = page.getContent();
long total = page.getTotalElements();
int totalPages = page.getTotalPages();
```

★FREQUENTLY ASKED Q20: What is the difference between findById() and findOne() / getByld()?

Answer:

- findById(id) — Returns Optional<T>; executes SELECT immediately; returns empty Optional if not found
- findOne(id) / getByld(id) — Returns a proxy (lazy reference); does not hit DB until fields are accessed; throws EntityNotFoundException if not found

findOne() is deprecated in newer Spring versions. Prefer findById() or getReferenceById() (Spring Data JPA 3+).

Q21: What is @Modifying and when is it required?

Answer:

@Modifying is used with @Query for UPDATE or DELETE JPQL/native queries. It tells Spring Data JPA that the query modifies data (not a SELECT).

```
@Modifying
@Transactional
@Query("UPDATE User u SET u.status = :status WHERE u.id = :id")
int updateUserStatus(@Param("id") Long id, @Param("status") String status);
```

Important: @Modifying queries require @Transactional (either on the method or the service class).

Q22: What is @EntityGraph and how does it work?

Answer:

@EntityGraph allows specifying which associations to eagerly load for a specific query, overriding the fetch type defined on the entity.

```
@Entity
@NamedEntityGraph(name = "User.orders",
    attributeNodes = @NamedAttributeNode("orders"))
public class User { ... }

// Repository
@EntityGraph(value = "User.orders")
Optional<User> findById(Long id);
```

Or use ad-hoc entity graph:

```
@EntityGraph(attributePaths = {"orders", "orders.items"})
List<User> findAll();
```

Section 5: Transactions in Spring Data JPA

★FREQUENTLY ASKED Q23: What is @Transactional and how does it work in Spring?

Answer:

@Transactional marks a method or class to execute within a transactional context. Spring uses AOP to begin a transaction before the method and commit or rollback after.

```
@Service
public class UserService {
    @Transactional
    public void transferFunds(Long fromId, Long toId, double amount) {
        // Both ops run in same transaction; rollback if exception thrown
    }
}
```

Key attributes:

- readOnly = true — Optimizes read-only operations (no dirty checking)
- rollbackFor = Exception.class — Triggers rollback on checked exceptions
- propagation — Defines how transactions propagate
- isolation — Sets transaction isolation level

★FREQUENTLY ASKED Q24: What are Transaction Propagation types?

Answer:

- REQUIRED (default) — Uses existing transaction or creates new one
- REQUIRES_NEW — Always creates a new transaction, suspends existing
- SUPPORTS — Uses existing transaction if present, runs non-transactionally otherwise
- NOT_SUPPORTED — Suspends existing transaction, runs non-transactionally
- MANDATORY — Must run within existing transaction; throws exception if none
- NEVER — Must NOT run in a transaction; throws exception if one exists
- NESTED — Runs within a nested transaction if one exists; creates new otherwise

★FREQUENTLY ASKED Q25: What are Transaction Isolation Levels?

Answer:

- READ_UNCOMMITTED — Can read uncommitted changes from other transactions (dirty reads allowed)
- READ_COMMITTED — Reads only committed data; prevents dirty reads
- REPEATABLE_READ — Same read returns same data within a transaction; prevents non-repeatable reads
- SERIALIZABLE — Full isolation; transactions run serially; prevents all anomalies but slowest

Default: Usually READ_COMMITTED (database default).

★FREQUENTLY ASKED Q26: Why does @Transactional not work when called from within the same class?

Answer:

This is a common interview trap. Spring's @Transactional uses AOP proxy-based advice. When a method calls another method in the SAME class, it bypasses the proxy and calls the target directly — so the transaction advice is never applied.

Solutions:

- Move the method to a separate bean/service class
- Self-inject the bean: @Autowired private MyService self; and call self.method()
- Use AspectJ mode instead of proxy mode (@EnableTransactionManagement(mode = AdviceMode.ASPECTJ))

Section 6: JPA Entity Lifecycle and Caching

★FREQUENTLY ASKED Q27: What are the four JPA entity states?

Answer:

- Transient — Object is created but not associated with EntityManager; no DB row
- Persistent (Managed) — Associated with EntityManager; changes tracked and synced to DB
- Detached — Was persistent but EntityManager closed or explicitly detached; changes not tracked
- Removed — Marked for deletion; will be deleted on transaction commit

Transitions: new → persist() → Persistent; Persistent → detach() → Detached; Persistent → remove() → Removed

Q28: What is the First-Level Cache (L1 Cache) in Hibernate/JPA?

Answer:

The first-level cache is the Persistence Context (EntityManager). It is enabled by default and scoped to the current transaction/session. Within the same transaction, the same entity is fetched only once from the database — subsequent fetches return the cached instance.

It cannot be disabled and is automatically cleared when the transaction ends.

Q29: What is the Second-Level Cache (L2 Cache)?

Answer:

The second-level cache is shared across sessions/transactions. It must be explicitly configured (e.g., EhCache, Caffeine, Hazelcast).

```
@Entity
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
```

```
public class Product { ... }
```

application.properties:

```
spring.jpa.properties.hibernate.cache.use_second_level_cache=true
spring.jpa.properties.hibernate.cache.region.factory_class=
    org.hibernate.cache.ehcache.EhCacheRegionFactory
```

Q30: What are JPA lifecycle callbacks?

Answer:

- `@PrePersist` — Called before entity is persisted (INSERT)
- `@PostPersist` — Called after entity is persisted
- `@PreUpdate` — Called before entity update (UPDATE)
- `@PostUpdate` — Called after entity update
- `@PreRemove` — Called before entity is deleted
- `@PostRemove` — Called after entity is deleted
- `@PostLoad` — Called after entity is loaded from DB

Example:

```
@PrePersist
public void onPrePersist() {
    this.createdAt = LocalDateTime.now();
}
```

Section 7: Spring Data JPA Auditing

★FREQUENTLY ASKED Q31: How do you implement Auditing in Spring Data JPA?

Answer:

Spring Data JPA provides built-in auditing support for tracking creation/modification timestamps and users.

Step 1: Enable auditing in configuration:

```
@EnableJpaAuditing
@SpringBootApplication
public class Application { }
```

Step 2: Annotate entity fields:

```
@Entity
@EntityListeners(AuditingEntityListener.class)
public class Product {

    @CreatedDate
    private LocalDateTime createdAt;

    @LastModifiedDate
    private LocalDateTime updatedAt;

    @CreatedBy
    private String createdBy;

    @LastModifiedBy
    private String modifiedBy;
```

```
}
```

Step 3: Implement AuditorAware for @CreatedBy / @LastModifiedBy:

```
@Bean
public AuditorAware<String> auditorProvider() {
    return () -> Optional.of(SecurityContextHolder.getContext()
        .getAuthentication().getName());
}
```

Section 8: Criteria API and Specifications

Q32: What is the Criteria API in JPA?

Answer:

The Criteria API provides a programmatic, type-safe way to build JPA queries using Java code instead of JPQL strings. It is useful for building dynamic queries.

```
CriteriaBuilder cb = entityManager.getCriteriaBuilder();
CriteriaQuery<User> cq = cb.createQuery(User.class);
Root<User> root = cq.from(User.class);
cq.select(root).where(cb.equal(root.get("status"), "ACTIVE"));
List<User> users = entityManager.createQuery(cq).getResultList();
```

★FREQUENTLY ASKED Q33: What is the Specification pattern in Spring Data JPA?

Answer:

Specification is a Spring Data JPA abstraction on top of the Criteria API. It lets you build reusable, composable query predicates.

Step 1: Repository extends JpaSpecificationExecutor:

```
public interface UserRepository extends JpaRepository<User, Long>,
    JpaSpecificationExecutor<User> { }
```

Step 2: Define Specifications:

```
public class UserSpecs {
    public static Specification<User> hasStatus(String status) {
        return (root, query, cb) -> cb.equal(root.get("status"), status);
    }
    public static Specification<User> olderThan(int age) {
        return (root, query, cb) -> cb.greaterThan(root.get("age"), age);
    }
}
```

Step 3: Compose and use:

```
List<User> users = repo.findAll(
    UserSpecs.hasStatus("ACTIVE").and(UserSpecs.olderThan(18))
);
```

Section 9: Projections and DTOs

★FREQUENTLY ASKED Q34: What are Projections in Spring Data JPA?

Answer:

Projections allow fetching only a subset of entity fields, improving performance by avoiding loading unnecessary data.

Interface-based Projection:

```
public interface UserSummary {
    String getFirstName();
    String getEmail();
}

List<UserSummary> findByStatus(String status);
```

Class-based Projection (DTO):

```
public class UserDTO {
    private String firstName;
    private String email;
    // constructor, getters
}

@Query("SELECT new com.example.UserDTO(u.firstName, u.email)
      FROM User u WHERE u.status = :status")
List<UserDTO> findUserDTOs(@Param("status") String status);
```

Q35: What is a Dynamic Projection in Spring Data JPA?**Answer:**

Dynamic projections let the caller specify the return type at call time using a generic type parameter.

```
// Repository method
<T> List<T> findByStatus(String status, Class<T> type);

// Usage
List<UserSummary> summaries = repo.findByStatus("ACTIVE", UserSummary.class);
List<User> fullUsers = repo.findByStatus("ACTIVE", User.class);
```

Section 10: Inheritance Mapping Strategies

★ FREQUENTLY ASKED Q36: What are the JPA Inheritance mapping strategies?**Answer:**

- SINGLE_TABLE — All classes in hierarchy stored in ONE table; discriminator column differentiates types; best performance but nullable columns
- TABLE_PER_CLASS — Each concrete class has its own table; no joins needed but difficult queries across hierarchy
- JOINED — Base class has its own table; subclasses have tables with only their fields; clean design but requires JOIN queries

Example (SINGLE_TABLE):

```
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "type")
```

```
public abstract class Vehicle { ... }

@Entity
@DiscriminatorValue("CAR")
public class Car extends Vehicle { ... }
```

Section 11: Custom Repository Implementations

★ **FREQUENTLY ASKED Q37: How do you add custom methods to a Spring Data JPA repository?**

Answer:

You can add custom behavior by creating a custom interface and an implementation class.

Step 1: Create custom interface:

```
public interface UserRepositoryCustom {
    List<User> findUsersWithComplexCriteria(String param);
}
```

Step 2: Implement it (class name must end with 'Impl' or configured suffix):

```
public class UserRepositoryImpl implements UserRepositoryCustom {
    @PersistenceContext
    private EntityManager em;

    public List<User> findUsersWithComplexCriteria(String param) {
        // use EntityManager for custom logic
    }
}
```

Step 3: Extend both in main repository:

```
public interface UserRepository extends JpaRepository<User, Long>,
    UserRepositoryCustom { }
```

Section 12: Performance and Best Practices

★ **FREQUENTLY ASKED Q38: What common performance pitfalls should you avoid in Spring Data JPA?**

Answer:

- N+1 Problem — Use JOIN FETCH or @EntityGraph to load associations in a single query
- Fetching entire entities when only a few fields are needed — Use projections or DTOs
- Using EAGER fetch globally — Set LAZY on associations; use JOIN FETCH when needed
- Not paginating large result sets — Always use Pageable for large data
- Unnecessary dirty checking — Use readOnly = true on read-only transactions
- Missing indexes on frequently queried columns — Always index foreign keys and search fields
- Selecting with SELECT * in native queries — Select only required columns

Q39: How do you enable SQL logging in Spring Data JPA for debugging?**Answer:**

In application.properties:

```
# Show SQL
spring.jpa.show-sql=true

# Format SQL for readability
spring.jpa.properties.hibernate.format_sql=true

# Show bind parameters (requires logging config)
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.type.descriptor.sql=TRACE
```

Q40: What is the difference between save() and saveAndFlush() in JPA Repository?**Answer:**

- save() — Persists the entity but the SQL might not be sent to DB immediately; waits until flush (end of transaction)
- saveAndFlush() — Persists AND immediately sends SQL to DB within the same transaction; useful when you need the data visible within the same transaction

Q41: How do you handle OptimisticLocking in Spring Data JPA?**Answer:**

Optimistic locking prevents concurrent updates by using a version field. JPA throws OptimisticLockException if the version doesn't match during update.

```
@Entity
public class Product {
    @Id private Long id;

    @Version
    private Long version; // auto-managed by JPA
}
```

On each UPDATE, the version is incremented. If two transactions try to update simultaneously, the second will fail with an exception.

Q42: What is the difference between @Lock and optimistic locking?**Answer:**

- Optimistic Locking (@Version) — No DB-level lock; detects conflict at commit time; good for low-contention scenarios
- Pessimistic Locking (@Lock) — Acquires DB lock at read time; prevents other transactions from reading/modifying

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
Optional<Product> findById(Long id);
```


Section 13: Schema Management and Configuration

★FREQUENTLY ASKED Q43: What are the values for spring.jpa.hibernate.ddl-auto?

Answer:

- none — Do nothing; schema is managed externally
- validate — Validate schema matches entities; throws exception if mismatch
- update — Update schema to match entities; adds new columns/tables but does NOT drop
- create — Create schema on startup; drops existing schema first
- create-drop — Create schema on startup; drop on application shutdown

Best practice: Use none or validate in production. Use update or create for development. Use Liquibase or Flyway for production migrations.

Q44: What is Flyway and how does it differ from Hibernate's DDL auto?

Answer:

Flyway is a database migration tool. Instead of letting Hibernate auto-generate DDL, Flyway manages versioned SQL migration scripts.

- Migration scripts are versioned: V1__create_users.sql, V2__add_email_index.sql
- Tracks applied migrations in flyway_schema_history table
- Provides rollback-safe, predictable schema changes
- Works independently of JPA/Hibernate

Best practice: Always use Flyway or Liquibase in production over ddl-auto=update.

Section 14: Advanced Topics

Q45: What is QueryDSL and how does it integrate with Spring Data JPA?

Answer:

QueryDSL is a framework for building type-safe queries using generated Q-classes. It provides a fluent API alternative to JPQL/Criteria API.

```
// Repository
public interface UserRepository extends QuerydslPredicateExecutor<User> { }

// Usage
QUser user = QUser.user;
Predicate predicate = user.email.endsWith("@example.com")
    .and(user.status.eq("ACTIVE"));
List<User> users = (List<User>) userRepo.findAll(predicate);
```

Q46: What is Spring Data REST and how does it work?

Answer:

Spring Data REST automatically exposes JPA repositories as RESTful endpoints over HTTP. It generates HATEOAS-compliant REST APIs without writing controllers.

```
// Just adding the dependency and this annotation is enough
@RepositoryRestResource(path = "users")
public interface UserRepository extends JpaRepository<User, Long> { }
```

This automatically creates: GET/POST /users, GET/PUT/DELETE /users/{id}

Q47: How do you handle soft deletes in Spring Data JPA?

Answer:

Soft delete marks a record as deleted without actually removing it from the database.

```
@Entity
@Where(clause = "deleted = false") // Hibernate specific
public class User {
    @Id private Long id;
    private boolean deleted = false;
}

// Custom delete implementation
@Modifying
@Query("UPDATE User u SET u.deleted = true WHERE u.id = :id")
void softDelete(@Param("id") Long id);
```

Q48: What is the difference between @NamedQuery and @Query?

Answer:

- @NamedQuery — Defined at entity class level; validated at startup; useful for reuse across DAOs
- @Query — Defined at repository method level; more convenient; validated at startup in Spring Data JPA

Both are validated at application startup, so query errors are caught early.

Q49: How do you implement batch inserts/updates for performance?

Answer:

application.properties:

```
spring.jpa.properties.hibernate.jdbc.batch_size=50
spring.jpa.properties.hibernate.order_inserts=true
spring.jpa.properties.hibernate.order_updates=true
```

Note: For batch inserts to work, use SEQUENCE or TABLE generation strategy, NOT IDENTITY (identity disables JDBC batching in Hibernate).

Q50: What is @PersistenceContext and how does it differ from @Autowired EntityManager?

Answer:

- @PersistenceContext — The JPA-standard way; injects a transactional/thread-safe

proxy for EntityManager; recommended

- @Autowired EntityManager — Spring-specific; in Spring Boot, also injects a thread-safe proxy, effectively equivalent

Prefer @PersistenceContext for JPA standard compliance.

Section 15: Scenario-Based Questions

★ FREQUENTLY ASKED Q51: How would you design a repository for a multi-tenant application?

Answer:

Approaches:

- Schema-per-tenant: Switch schemas dynamically using AbstractRoutingDataSource
 - Row-level tenancy: Add tenantId to all entities and filter via @Filter or Specifications
 - Use Hibernate's built-in multi-tenancy support with TenantIdentifierResolver and MultiTenantConnectionProvider
-

Q52: How would you implement a full-text search with Spring Data JPA?

Answer:

- Use LIKE queries for simple cases: findByNameContainingIgnoreCase(String keyword)
 - Use database full-text search: @Query with native SQL MATCH AGAINST (MySQL)
 - Integrate Hibernate Search (Lucene/Elasticsearch) for advanced full-text search
 - Use Spring Data Elasticsearch as a separate data store for search
-

★ FREQUENTLY ASKED Q53: How do you test Spring Data JPA repositories?

Answer:

Use @DataJpaTest which configures an in-memory H2 database, loads only JPA-related beans.

```
@DataJpaTest
class UserRepositoryTest {
    @Autowired
    private UserRepository userRepository;

    @Test
    void testFindByEmail() {
        User user = new User("john@example.com", "John");
        userRepository.save(user);
        Optional<User> found = userRepository.findByEmail("john@example.com");
        assertThat(found).isPresent();
    }
}
```

★ FREQUENTLY ASKED Q54: What is a LazyInitializationException and how do you fix

it?

Answer:

LazyInitializationException occurs when you try to access a LAZY-loaded association outside of a Hibernate session (transaction boundary).

Fixes:

- Use JOIN FETCH or @EntityGraph to load the association within the transaction
- Add @Transactional to the calling service method so the session remains open
- Use spring.jpa.open-in-view=true (Open Session in View — NOT recommended for production; hides layering issues)
- Use DTO projections to avoid loading associations entirely

Q55: What is the difference between merge() and persist() in JPA?

Answer:

- persist() — Makes a new transient entity managed; entity must be new (not exist in DB)
- merge() — Merges state of a detached entity into a new managed instance; works for both new and existing entities

Spring Data's save() method: calls persist() for new entities (no ID), calls merge() for existing entities (has ID).

Spring Data JPA — Quick Reference Cheat Sheet

Core Repository Hierarchy

Interface	Key Methods / Purpose
Repository<T, ID>	Marker interface — no methods
CrudRepository<T, ID>	save, findById, findAll, deleteById, count, existsById
PagingAndSortingRepository	findAll(Pageable), findAll(Sort)
JpaRepository<T, ID>	flush, saveAndFlush, deleteInBatch, getReferenceById
JpaSpecificationExecutor	findAll(Spec), findOne(Spec), count(Spec)

Common Annotations

Annotation	Purpose / Usage
@Entity	Marks class as JPA entity
@Table(name)	Maps entity to specific table
@Id	Marks primary key
@GeneratedValue	Strategy: IDENTITY, SEQUENCE, TABLE, AUTO
@Column	name, nullable, length, unique, updatable
@Transient	Field not persisted
@Embedded	Embeds @Embeddable object
@OneToMany	mappedBy, cascade, fetch, orphanRemoval
@ManyToOne	fetch, cascade, optional
@ManyToMany	mappedBy, cascade, @JoinTable
@Version	Optimistic locking version field
@Query	Custom JPQL or native query
@Modifying	Required for UPDATE/DELETE @Query
@Transactional	Transaction management on method/class
@EntityGraph	Override fetch strategy per query
@Lock	Pessimistic locking on query
@CreatedDate	Auto audit: creation timestamp
@LastModifiedDate	Auto audit: update timestamp

Query Method Keywords

Keyword / Pattern	Generated SQL equivalent
findBy / readBy / getBy	SELECT ... WHERE
And / Or	AND / OR
Is, Equals	= ?
Between	BETWEEN ? AND ?
LessThan / GreaterThan	< ? / > ?
Containing / Like	LIKE '%?%' / LIKE ?
StartingWith / EndingWith	LIKE '?%' / LIKE '%?'
In / NotIn	IN (?) / NOT IN (?)
IsNull / IsNotNull	IS NULL / IS NOT NULL
True / False	= true / = false
OrderBy...Asc/Desc	ORDER BY ... ASC / DESC
countBy...	SELECT COUNT(*)
deleteBy...	DELETE WHERE

Keyword / Pattern	Generated SQL equivalent
existsBy...	SELECT CASE WHEN COUNT > 0

Transaction Propagation Quick Reference

Propagation	Behavior
REQUIRED	Join existing TX or create new (default)
REQUIRES_NEW	Always create new TX; suspend existing
SUPPORTS	Use existing TX if present; else non-TX
NOT_SUPPORTED	Suspend TX; run non-TX
MANDATORY	Must have existing TX; else exception
NEVER	Must NOT be in TX; else exception
NESTED	Create savepoint within existing TX

application.properties Key Settings

Property	Recommended Value
spring.jpa.hibernate.ddl-auto	validate (prod), create (dev)
spring.jpa.show-sql	true (dev only)
spring.jpa.open-in-view	false (recommended)
hibernate.jdbc.batch_size	20-50 for batch ops

N+1 Problem — Quick Fix Reference

Problem	Solution
N+1 on @OneToMany	@Query with JOIN FETCH
N+1 on @ManyToOne	JOIN FETCH or @BatchSize(size=20)
Multiple associations	@EntityGraph(attributePaths = {...})
Avoid entity loading	Use Interface / DTO projections
Read-only queries	@Transactional(readOnly=true)